Bilingual Fact-Checker

Project 1 explained: how it works, what runs when, the local LoRA judge, results, and how to explain it to anyone.

Explain it in 30 seconds

It is a multi-agent AI system that checks whether a claim is **TRUE**, **FALSE** or **NOT ENOUGH INFO**, in English or Arabic. A team of specialised agents searches Wikipedia, a local knowledge base and research papers, quotes the evidence, and a judge decides using **only** evidence that was actually retrieved. Every citation is checked against the real text, so the system cannot invent sources, and a human approves every report before it is published. I measure it against a single-agent baseline on the FEVER benchmark: accuracy, evidence quality, fake citations, cost and speed. There is also an optional local judge: a small 4B model fine-tuned with LoRA so the decision step can run privately and for free on my own machine.

Contents

1. What the project does
2. The big picture: two ways to check a claim
3. The agent team, step by step
4. The single-agent baseline
5. Where the data comes from
6. The local knowledge base
7. Which model is used when
8. The local LoRA judge
9. Guardrails
10. How it is evaluated, and results so far
11. Commands cheat sheet
12. Status and next steps
13. Questions people will ask (with answers)
14. Glossary

Current setup: every LLM step runs on **Qwen3-235B** (`qwen.qwen3-235b-a22b-2507`) through **Amazon Bedrock** (us-east-1). Figures in this document are from the runs made on 12 November 2026.

1. What the project does

You give it a claim (or a short article containing several claims). It returns a verdict per claim, a short explanation in your language, and citations that point to exact sentences in real sources.

Example (real output)

```
$ make check CLAIM="The Chaperone (film) was directed by Michael Bay only."

FALSE (confidence 1.00)
The 2011 film The Chaperone was directed by Stephen Herek, not Michael Bay.
- "The Chaperone is a 2011 American comedy film directed by Stephen Herek, and
   also produced by WWE Studios." -- The Chaperone (film), sentence 0
1 claim . 1 round . 4 LLM calls . 1 tool call . $0.001
publish? [a]pprove / [e]dit / [r]eject
```

The three possible answers

Label	Meaning	Rule the model must follow
TRUE	The evidence supports the claim	The evidence states **every** part of the claim. A partial match is not enough.
FALSE	The evidence contradicts the claim	The evidence says something that cannot be true at the same time (e.g. a different date for the same event).
NOT ENOUGH INFO	The evidence does not settle it	The default. Used when any part is not mentioned, when two descriptions could both be true, or when deciding would need the model's own memory.

> **Key idea to say out loud:** "Not mentioned is not the same as contradicted." The system refuses to use the model's own memory; if the evidence is not there, the answer is NOT ENOUGH INFO.

What makes it different from just asking a chatbot

- **Verified citations:** every quote is matched against the sentence that was really retrieved. Invented quotes are thrown away and counted.
- **Evidence only:** the judge sees nothing but the verified evidence, numbered, and can only cite by number.
- **Human in the loop:** nothing is published without your approve / edit / reject.
- **Bilingual:** Arabic claims are searched in Arabic and English, and the report is written in the claim's language.
- **Measured:** every design choice is compared on a fixed benchmark (accuracy, evidence quality, fake citations, cost, time).

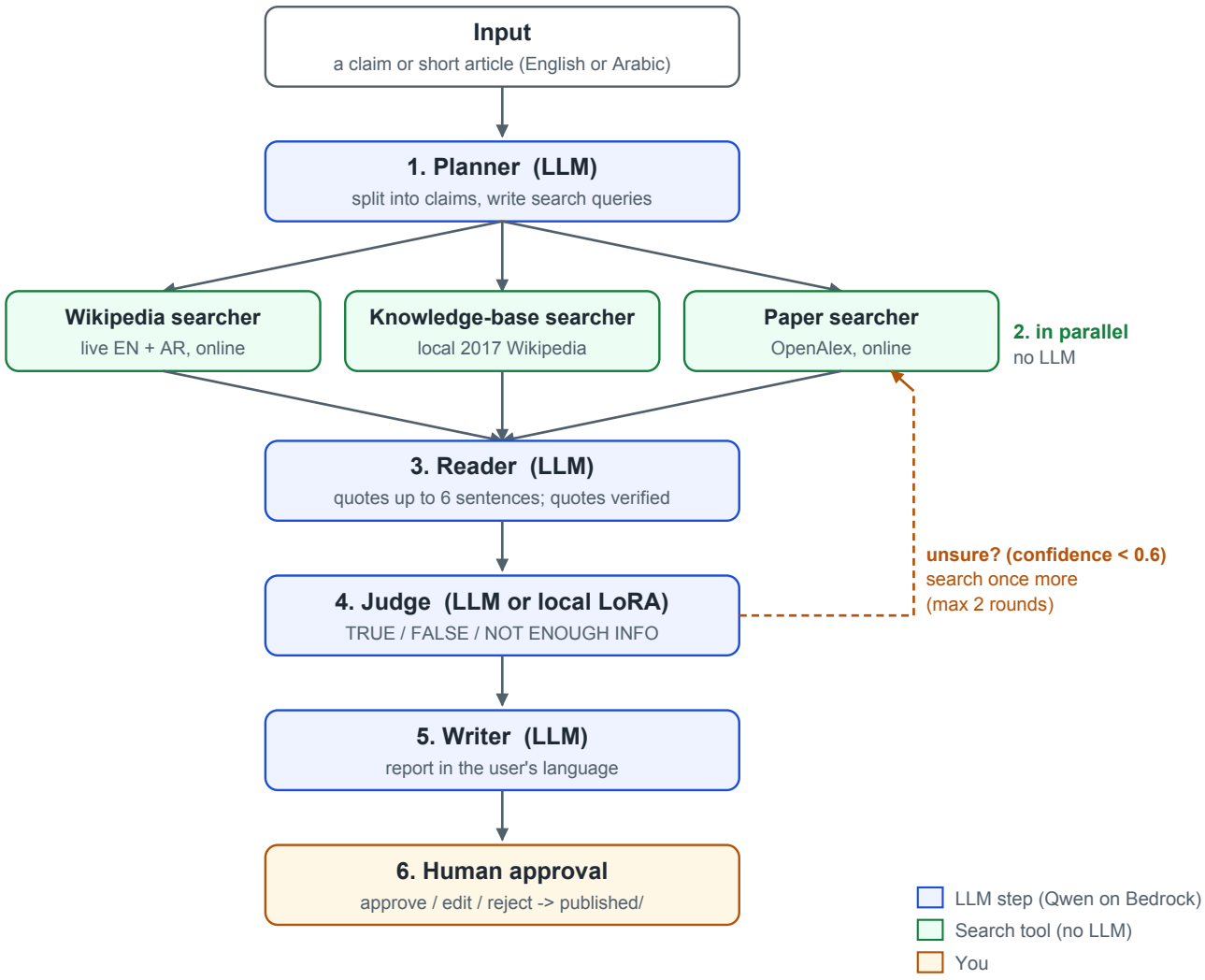
2. The big picture: two ways to check a claim

The project contains two systems that use the same tools, so they can be compared fairly:

	Agent team (the main system)	Single agent (the baseline)
Idea	Several specialised agents, each with one job	One agent does everything in a loop
Search	3 searchers run **at the same time**	Picks one tool per step, up to 8 steps
Decision	A separate judge decides from verified quotes	The same agent decides at the end
If unsure	Judge asks for one more search round	Keeps searching until its step limit
Command	`make check`	`make check-single`
Built with	LangGraph (graph of steps, parallel branches, pause for approval)	A plain Python ReAct loop

Why have both? The single agent is the **control group**. The team is more complex and slower, so it has to prove it is worth it. On 100 claims both reach 69% accuracy, but the team's answers are backed by the right evidence far more often (FEVER score 52% vs 7%) and it invents far fewer citations (2.6% vs 19.3%).

The agent team at a glance



3. The agent team, step by step

Step	Job	Gets	Gives back	Uses
1. Planner	Split the input into up to 5 checkable claims; write 1-3 search queries per claim (Arabic + English for Arabic input)	Your text	Claims + queries (JSON)	LLM
2. Searchers	Fetch documents. Wikipedia, knowledge base and papers run in parallel for every claim	Queries	Documents as numbered sentences	Tools, no LLM
3. Reader	Pick up to 6 sentences that support, refute or give context; quotes are verified by code	Up to 60 most relevant sentences	Verified quotes with a stance	LLM + code check
4. Judge	Decide the label from the verified quotes only; give confidence; ask for more search if unsure	Numbered evidence	Label, confidence, evidence numbers	LLM or LoRA
5. Writer	Headline + 1-3 sentence explanation per claim, in the user's language	Verdicts + evidence	Report text	LLM
6. Approval	You approve, edit or reject; approved reports go to published/	Report	Decision	You

Each step in a bit more detail

- **Planner.** If the input is a single claim it is kept unchanged. Opinions and questions are ignored. It also writes an English version of each claim, so the judge always sees English even for Arabic input.
- **Searchers.** Wikipedia: live search + page fetch, English and Arabic. Knowledge base: local search over 11,172 chunks of 2017 Wikipedia. Papers: OpenAlex abstracts, meant for scientific or medical claims. Online answers are cached in `data/cache/`, so reruns are cheap and repeatable.
- **Reader.** Code first keeps the 60 sentences that share the most words with the claim (plus each page's first sentence). The LLM then quotes up to 6 and marks each as supports, refutes or context. A quote must match its real sentence (85% similarity) or it is dropped and counted as a fake citation.
- **Judge.** Sees a numbered list like `0. [refutes] (The Chaperone) ...`. It answers with a label, a confidence from 0 to 1, the evidence numbers it relies on, and an explanation. Numbers that do not exist are dropped. If confidence is below 0.6 it writes up to 2 follow-up queries and the searchers run one more round (2 rounds maximum).
- **Writer.** Writes for a general reader. The citations in the final report are added by code from verified evidence; the writer never types a citation itself.
- **Approval.** LangGraph pauses the graph (an interrupt) and waits for you. Benchmarks skip this step.

> **When does the judge come in?** Only after evidence has been collected **and** verified. The judge never searches and never sees raw web pages, only the reader's checked quotes. That is what keeps its citations honest.

Every agent answers in validated JSON (Pydantic models such as `PlannerOut`, `ReaderOut`, `JudgeOut`). If a reply does not fit the format, it is retried once; if it fails again the claim is marked ERROR. This is where the current 'reader crashes' come from (see section 12).

4. The single-agent baseline

One ReAct agent ("reason + act"): on each step it writes a short thought and calls one tool, reads the result, and repeats. After at most 8 steps, or when it chooses finish, it gives a verdict with up to 3 citations. The same label rules and citation checks apply, but the checks happen **after** it has decided.



Why it scores well on accuracy but badly on evidence: it often gets the right label but cites the wrong sentence, or a sentence it never read. That is a sign it is partly answering from the model's memory, which is exactly what the team design prevents.

5. Where the data comes from

There are two different kinds of data, and it helps to keep them apart when explaining:

- **The claims to check** (with the correct answers) come from public datasets, stored as files.
- **The evidence** the agents search comes from the knowledge base (local) and from Wikipedia and OpenAlex (online).

The claims (questions with known answers)

File	What is in it	Used for
data/fever_300.jsonl	300 English claims from the FEVER dev set: 100 TRUE, 100 FALSE, 100 NOT ENOUGH INFO (seed 42). Each line also stores the correct label and the gold evidence sentences.	make bench (default)
data/arafacts_sample.jsonl	125 Arabic claims from AraFacts: 50 TRUE, 50 FALSE, 25 unverifiable	make bench DATASET=arafacts
data/fever_train_sample.jsonl	4,500 different FEVER claims, no overlap with the test set	Training the LoRA judge only

The labels in `fever_300.jsonl` are interleaved (TRUE, FALSE, NEI, TRUE, ...), so `LIMIT=30` gives 10 of each and `LIMIT=100` gives 34 / 33 / 33. Single and team always get exactly the same claims.

The evidence sources

Searcher	Source	Online?	Good for
kb	Local copy of 5,159 Wikipedia pages from 2017, in data/kb/	No	FEVER claims (the gold pages are in it); fast, free
wiki	Live Wikipedia, English and Arabic (MediaWiki API)	Yes	Current facts and Arabic claims
papers	OpenAlex research paper abstracts	Yes	Scientific and medical claims

There is **no general web search** (no Google or Bing). If nothing useful is found, the judge sees "(no evidence found)", may trigger one more search round, and otherwise answers NOT ENOUGH INFO.

What is in the data/ folder

Folder	Size	What it is
raw/	1.7 GB	Original downloads: FEVER dev + train sets, the AraFacts CSV, the 2017 Wikipedia dump (wiki-pages.zip). Source material only.

Folder	Size	What it is
kb/	7.7 MB	The local knowledge base built from the dump: pages.jsonl, chunks.jsonl and index metadata.
cache/	8.6 MB	Saved answers from live Wikipedia and OpenAlex. Delete cache/http/ to fetch fresh copies.
lora/	14 MB	Training data for the LoRA judge (train / val / judge_eval). Not used by make bench.

6. The local knowledge base

Built once with `make kb` (or `make kb-bm25` without embeddings) from the 1.7 GB FEVER Wikipedia dump of 5.4 million pages. It keeps only what is needed:

- **Gold pages:** the pages that hold the evidence for the 300 test claims.
- **Hard distractors:** about 2,000 pages whose title appears in a test claim (similar names, wrong page).
- **Easy distractors:** 3,000 random pages.
- Result: **5,159 pages** split into **11,172 chunks** of 3 sentences each.

How it searches

Method	How it works	Status
BM25	Classic keyword search: ranks chunks by shared, rare words	**Active.** Finds the correct page in the top 3 for 95% of claims
Embeddings	Meaning-based search with baai/bge-m3 vectors (via OpenRouter, about $0.02 to build)	Not built yet
Hybrid (RRF)	Merges BM25 and embedding rankings (Reciprocal Rank Fusion)	Used automatically once embeddings exist
LLM rerank	An LLM reorders the top 20 results	Optional (KB_RERANK=llm)

Honest caveat: the knowledge base is "closed world": it is guaranteed to contain the right pages. It measures retrieval and reasoning, not open-web search. Live Wikipedia is there for the real-world case.

7. Which model is used when

Part	Model	Runs where	When
Planner, reader, judge, writer	Qwen3-235B (qwen.qwen3-235b-a22b-2507)	AWS Bedrock, us-east-1	Every team check / team benchmark
Single agent	Same Qwen3-235B	AWS Bedrock	check-single, bench MODE=single
Judge with JUDGE=llm:pro	Whatever BEDROCK_MODEL_PRO is. Today also Qwen3-235B, so **no change**	AWS Bedrock	Only if you set a different PRO model
Judge with JUDGE=lora:factjudge	Qwen3-4B + your LoRA ("factjudge")	**Your Mac**, Ollama	After training (section 8)
Base small judge (for comparison)	Qwen3-4B without LoRA	Your Mac, Ollama	make judge-eval only
Knowledge-base embeddings	baai/bge-m3	OpenRouter	make kb, and hybrid search queries
Searchers	No model	Local + Wikipedia / OpenAlex APIs	Every check

All of this is set in `.env`: `LLM_BACKEND=bedrock`, `BEDROCK_MODEL_FLASH` (normal steps) and `BEDROCK_MODEL_PRO` (the pro tier). The code also supports OpenRouter, Google Vertex AI and Ollama as backends, so the model can be swapped without code changes. Bedrock's effort setting only applies to Claude models, so it has no effect on Qwen.

8. The local LoRA judge

What LoRA is, in one sentence

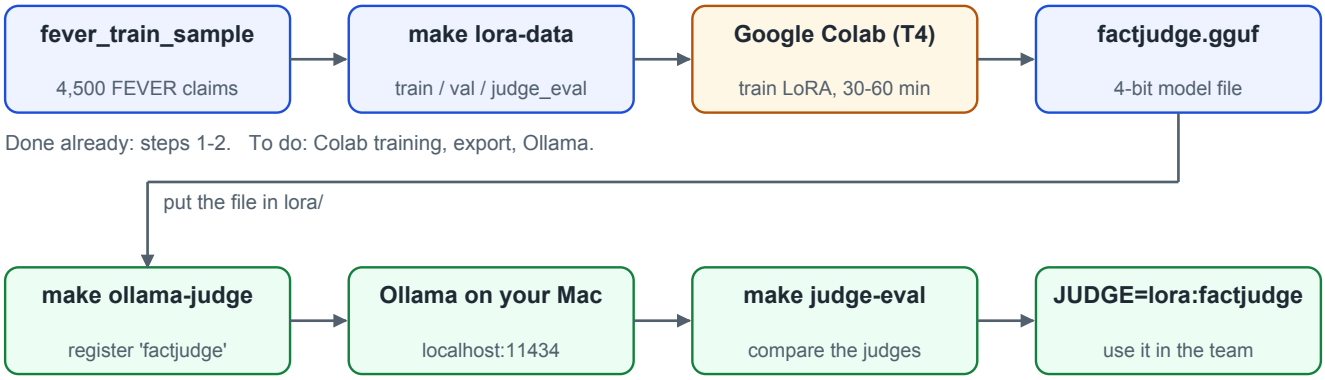
LoRA (Low-Rank Adaptation) fine-tunes a model by training small add-on weight matrices instead of the whole model, so you can teach a model a narrow job on a free GPU in under an hour.

Why the project uses it

	Cloud judge (now)	LoRA judge (planned)
Model	Qwen3-235B on Bedrock	Qwen3-4B + LoRA, about 60 times smaller
Runs on	AWS, over the internet	Your Mac, through Ollama
Cost per judgment	Paid (small)	Free
Claim and evidence leave your machine?	Yes	No (for the judge step)
Trained for this exact job?	No, general purpose	Yes, on 4,255 labelled FEVER examples

The question it answers: **can a small model trained for one job match a huge general model at that job?** It may also help with NOT ENOUGH INFO, the current weak spot, because it learns from many labelled examples of it.

The pipeline



Step	Command / place	What happens
1. Build data	make lora-data (done)	From the 4,500 training claims: train.jsonl (4,255), val.jsonl (223), and judge_eval.jsonl (297 test claims with fixed evidence). Labels are balanced.
2. Train	lora/train_judge.ipynb in Google Colab, free T4 GPU	Loads unsloth/Qwen3-4B-Instruct-2507 in 4-bit, adds LoRA (rank 16) to the attention and MLP layers, trains 1 epoch on the answer only, checks validation accuracy. About 30-60 min.
3. Export	Last notebook cell	Saves factjudge-Q4_K_M.gguf (4-bit quantised). Download it and put it in lora/.
4. Install	Ollama, once for all projects	make ollama-base pulls the plain Qwen3-4B for comparison; make ollama-judge registers your model as 'factjudge' using lora/Modelfile (temperature 0).
5. Compare	make judge-eval	Runs the cloud judge, base Qwen3-4B and factjudge on the same 297 claims with the same evidence, so differences come from the judge alone.
6. Use	make bench MODE=team JUDGE=lora:factjudge	The full team, with only the judge step running locally.

How the training examples are built

- **TRUE / FALSE examples:** the gold evidence sentences plus up to 2 other sentences from the same page, so the model has to find the relevant line rather than trust everything it is shown.
- **NOT ENOUGH INFO examples:** the first sentences of a page about the same entity: the subject is there, the fact is not.
- Exactly the same prompt format is used in training and at inference time (`factcheck/judge_lora.py`). That match is what makes the fine-tune work; change the format and you must retrain.

Limits to mention

- It returns only a label, with no confidence, so confidence is fixed at 1.0 and the "search again" round never triggers.
- Only the judge runs locally. The planner, reader and writer still call Bedrock, and evidence is still fetched online.
- Status: data is built; training, Ollama install and judge-eval have not been run yet.

9. Guardrails

In place

Guardrail	What it prevents	Where
Citation verification	Invented quotes and sources: every quote must match a sentence that was really retrieved	factcheck/citations.py
Judge cites by number only	The judge pointing to evidence that does not exist	factcheck/team.py (judge)
Citations rendered by code	The writer inventing references in the final report	factcheck/team.py (report)
Validated JSON + one retry	Malformed or unexpected model output	agentcore/llm.py
Evidence-only label rules	Answers from the model's memory; guessing instead of NOT ENOUGH INFO	factcheck/schemas.py (LABEL_RULES)
Human approval	Anything being published without a person saying yes	agentcore/approval.py
Hard limits	Runaway loops and cost: 5 claims, 8 single-agent steps, 2 search rounds, 6,000 characters per tool result	team.py, baseline.py
Read-only tools	Agents changing or sending anything	factcheck/tools/

Not yet in place

- **Prompt-injection protection:** text from web pages goes into prompts without being marked as untrusted data.
- **Input checks:** no length limit and no rejection of empty or off-topic input.
- **Cost cap:** step limits exist, but no maximum dollar amount per run.
- **Writer check:** nothing verifies that the writer added no new facts.

10. How it is evaluated, and results so far

Layer	Command	What it measures
End-to-end benchmark	make bench, make compare	Accuracy, per-label F1, FEVER score, gold page cited, fake citations, cost, time, errors
Retrieval	make kb-eval	Does search find the right page / evidence in the top 1, 3, 5, 10 results
Judge only	make judge-eval	Judges compared on the same fixed evidence (for the LoRA judge)
Unit tests	make test	38 offline tests with a fake LLM: no network, no cost

What the benchmark columns mean

Metric	Meaning
accuracy	Share of claims with the correct label
macro-F1	Accuracy averaged fairly over the 3 labels, so a weak label is not hidden
FEVER score	Label correct **and** a complete set of gold evidence sentences cited (knowledge-base citations only). The official FEVER metric and the strictest number.
gold page cited	At least one correct Wikipedia page is cited
fake citations	Citations to text the agent never actually read
errors	Claims that crashed; they count as wrong

Results: 100 FEVER claims, Qwen3-235B, all searchers

	Single agent	Agent team
Accuracy	69.0%	69.0%
macro-F1	0.637	**0.663**
FEVER score	7.0%	**52.0%**
Gold page cited	67.2%	**80.6%**
Fake citations	19.3%	**2.6%**
Cost per claim	$0.0021	**$0.0017**
Time per claim	**8.9 s**	16.9 s
Errors	**0**	4 (all in the reader step)

Correct label	Single got right	Team got right
TRUE	31 / 34	33 / 34
FALSE	31 / 33	27 / 33 (+3 crashes)
NOT ENOUGH INFO	7 / 33	9 / 33 (+1 crash)

How to read this: same accuracy, very different trustworthiness. The team gets the label right and cites the real proof far more often, at the same cost. Its two weaknesses are the 4 reader crashes and NOT ENOUGH INFO.

Effect of the label rules (30 claims, before and after)

	Team before	Team after	Single before	Single after
Accuracy	56.7%	70.0%	63.3%	66.7%
FEVER score	40.0%	50.0%	3.3%	3.3%
NOT ENOUGH INFO right	0 / 10	3 / 10	1 / 10	1 / 10

With 100 claims, one claim is 1 percentage point, so small differences can be noise. Retrieval (BM25 only, 200 claims): correct page in top 1 for 76%, top 3 for 95%, top 10 for 98%.

11. Commands cheat sheet

Run from the `1-fact-checker` folder. `make` is a shortcut runner: `make bench MODE=team LIMIT=30` expands to `uv run python -m factcheck bench --mode team --limit 30 ...`. Add `-n` to print the real command without running it.

Command	What it does
`make setup`	Install dependencies, create .env and prices.toml
`make test`	38 offline tests (free)
`make smoke`	One LLM call to check the key and model
`make data`	Build the FEVER test / train samples and the AraFacts sample
`make kb / make kb-bm25`	Build the knowledge base with / without embeddings
`make kb-eval`	Retrieval recall
`make check CLAIM="..."`	Agent team -> report -> your approval -> published/
`make check-single CLAIM="..."`	Single-agent baseline on one claim
`make bench MODE=single|team LIMIT=30`	Benchmark on the first N claims (no LIMIT = all 300)
`make bench MODE=team DATASET=arafacts`	Arabic benchmark (125 claims)
`... SEARCHERS=kb`	Use only some searchers, e.g. local knowledge base only
`... JUDGE=llm:pro / JUDGE=lora:factjudge`	Use the PRO model or the local LoRA model as judge
`make compare`	Table of all runs -> results/RESULTS.md and the README
`make lora-data`	Build LoRA training data
`make ollama-base / make ollama-judge`	Pull the base 4B model / register your trained model in Ollama
`make judge-eval`	Compare judges on the same evidence
`make trace RUN=<run_id>`	Summarise one run: every LLM and tool call with tokens, cost and time

Where results go

- `results/<run_id>.jsonl`: one line per claim, written as each finishes (a crash loses nothing).
- `results/<run_id>.summary.json`: the totals for that run.
- `traces/<run_id>.jsonl`: every LLM and tool call, for debugging.
- `published/`: reports you approved.

12. Status and next steps

Area	Status
Code for all 12 build steps	Done; 38 offline tests pass
Live single-claim checks (EN + AR)	Verified
Benchmarks	30- and 100-claim runs done; full 300-claim runs not yet
Arabic benchmark	Only 15 of 125 claims run
Knowledge-base embeddings	Not built (BM25 only)
LoRA judge	Data built; training, Ollama and judge-eval not done
Live approval step	Tested offline; needs a run in an interactive terminal
Demo video	Not recorded
README	Partly out of date (still mentions OpenRouter / Gemini as the main models)

Recommended order

- **1. Fix the reader crashes** (Qwen sometimes stops partway through its JSON). Worth about 3-4% accuracy.
- **2. Improve NOT ENOUGH INFO**, e.g. make the judge check each part of the claim before choosing a label.
- **3. Full benchmarks:** 300 claims single and team, then AraFacts; update the README with make compare.
- **4. LoRA judge:** train in Colab, install Ollama, run judge-eval.
- **5. Guardrails and evals:** mark retrieved text as untrusted, input checks, cost cap, a regression check, repeated runs to measure noise.
- **6. Demo video** (4 scenes listed in the README).

13. Questions people will ask (with answers)

Q: Why several agents instead of one?

Splitting the work lets each step be checked. The reader's quotes are verified before the judge sees them, and the judge can only cite by number. With one agent, searching, reading and deciding are mixed together and nothing stops it from citing what it never read. The data shows it: same accuracy, but 2.6% fake citations for the team versus 19.3% for the single agent.

Q: How do you stop it from hallucinating sources?

Three layers: every retrieved sentence is stored with its source and number; every quote must match its sentence (85% similarity) or it is dropped; the judge cites evidence by number and the final citations are rendered by code, not written by the model.

Q: Then why is the accuracy the same as the single agent?

Accuracy only checks the label. The single agent often gets the label right for the wrong reason, partly from the model's memory. The FEVER score, which also checks the evidence, is 52% for the team and 7% for the single agent.

Q: What is the weakest part?

NOT ENOUGH INFO: the model tends to answer TRUE or FALSE when the evidence does not settle the claim. Explicit label rules lifted the team from 0/10 to 3/10 on 30 claims; it is still the main thing to improve. The other issue is occasional cut-off JSON from the reader.

Q: Why FEVER, and why a 'closed-world' knowledge base?

FEVER is the standard fact-verification benchmark with labelled claims and gold evidence sentences, so evidence quality can be scored, not just labels. The knowledge base contains the gold pages plus distractors, so it tests retrieval and reasoning reproducibly. Live Wikipedia covers the real-world case.

Q: Why Qwen on Bedrock and not Claude or GPT?

It is the strong model available on my AWS account; in a comparison of Bedrock models it gave the best balance of accuracy, fake citations and cost (about $0.0017 per claim). The model is set in .env, and the code supports Bedrock, OpenRouter, Vertex AI and Ollama.

Q: What does LangGraph do here?

It runs the team as a graph: the searchers run in parallel (Send), the judge can loop back for another search round, and the graph pauses for human approval (interrupt) and resumes after your decision.

Q: Why train a LoRA model?

To test whether a small, specialised 4B model running locally can match a large cloud model on the judging step: free, private and fast. judge-eval compares them on identical evidence so the comparison is fair.

Q: How is Arabic handled?

The planner writes Arabic and English queries, the Wikipedia searcher uses Arabic and English Wikipedia, the judge sees an English version of the claim, and the writer answers in the claim's language. It is evaluated on the AraFacts dataset.

Q: How much does it cost?

About $0.0017 per claim for the team on Qwen3-235B; a full 300-claim run is roughly $0.50. Every call is traced with tokens, cost and time.

Q: What would you add for production?

A general web searcher for news claims, prompt-injection protection on retrieved text, input checks, a cost cap, an automatic regression check in CI, and the local LoRA judge for privacy.

14. Glossary

Term	Meaning
Agent	An LLM given one job, tools and a fixed output format
ReAct	"Reason + act": think, call a tool, read the result, repeat (used by the single agent)
LangGraph	Library for building agent workflows as graphs with parallel branches, loops and pauses
Structured output	The model must answer in JSON that matches a schema (Pydantic); invalid replies are retried
BM25	Keyword search that ranks text by shared, rare words
Embeddings	Numbers that represent meaning, so similar texts can be found even with different words
RRF	Reciprocal Rank Fusion: merges two ranked lists (BM25 + embeddings) into one
Rerank	A second, smarter pass that reorders the top search results
FEVER	Fact Extraction and VERification: a public benchmark of claims labelled against Wikipedia
FEVER score	Correct label plus a complete set of correct evidence sentences cited
AraFacts	A public dataset of Arabic claims checked by fact-checkers (non-commercial licence)
macro-F1	Average of the per-label F1 scores, so each label counts equally
LoRA	Low-Rank Adaptation: fine-tuning by training small add-on weights instead of the whole model
GGUF / Q4_K_M	A file format for running models locally; Q4_K_M means compressed to about 4 bits per weight
Ollama	A program that runs local models on your computer behind a simple API (localhost:11434)
Amazon Bedrock	AWS service that hosts many LLMs behind one API
Trace	A log line for every LLM and tool call: tokens, cost, time, result
Human-in-the-loop	A person must approve before an action (here: publishing a report)